

PREFLOW: Safe preemption for LLM prefill

Preemption fixes head-of-line blocking. It also decides who pays for the queue. Every time a scheduler moves a short prompt forward, some other prompt waits longer. Under shortest-remaining-processing-time (SRPT), that trade can be excellent for the mean and brutal for an individual long request.

We see both sides in our synthetic long-context stress test. SRPT improves mean time to first token (TTFT) by 84.6% over FCFS, but the worst matched request takes $7.52\times$ as long as it does under FCFS. PREFLOW with 120% slack retains a 78.2% mean improvement and limits the observed worst matched slowdown to $2.37\times$. With 30% slack, it still improves mean TTFT by 68.2%, while the observed worst slowdown is $1.37\times$.

Can we keep most of the benefit of preemption without arbitrarily sacrificing long requests?

PREFLOW separates that question into two decisions. A shortest-work objective decides which request should run next. A separate feasibility test decides whether allowing that action is still safe for every request already in the system. High-value work can overtake, but only while the queue has enough slack to absorb the delay.

The synthetic stress test

The workload is deliberately severe. It mixes prompt lengths that differ by $25\times$ and sends them through bursty arrival phases so that queue ordering has a visible effect. It is not a persistent-overload experiment: queues build during bursts and later drain.

- The trace contains 1,000 requests generated by a Markov-modulated Poisson process, averaging 0.5 requests/s over a 2,000-second arrival span.
- Prompt buckets are 4K, 8K, 16K, 32K, 65K, and 100K tokens, with shares of 30%, 25%, 20%, 14%, 8%, and 3%.
- Each request asks for 128 output tokens.
- One TP=2 prefill instance and one TP=2 decode instance run across four Ascend NPUs. Prefill uses fixed 4,096-token chunks, eight resident requests, and one scheduled request per step.
- The model is Qwen3-30B-A3B-Instruct-2507 with dummy weight loading. Prefix caching is disabled.

The six prompt classes span $25\times$ in token count, but FCFS compresses their mean TTFT into a narrow 116.6–123.8 second range. This does not mean their execution costs are equal. During bursts, queueing dominates total TTFT strongly enough to hide much of the underlying cost difference.

PREFLOW-50 lets short requests pass while earlier deadlines still have enough slack. Mean TTFT falls to 5.0 seconds for 4K prompts and then grows with

prompt size, reaching 186.9 seconds at 100K. The grouped bars use the observed wall-clock TTFTs from the same matched benchmark requests.

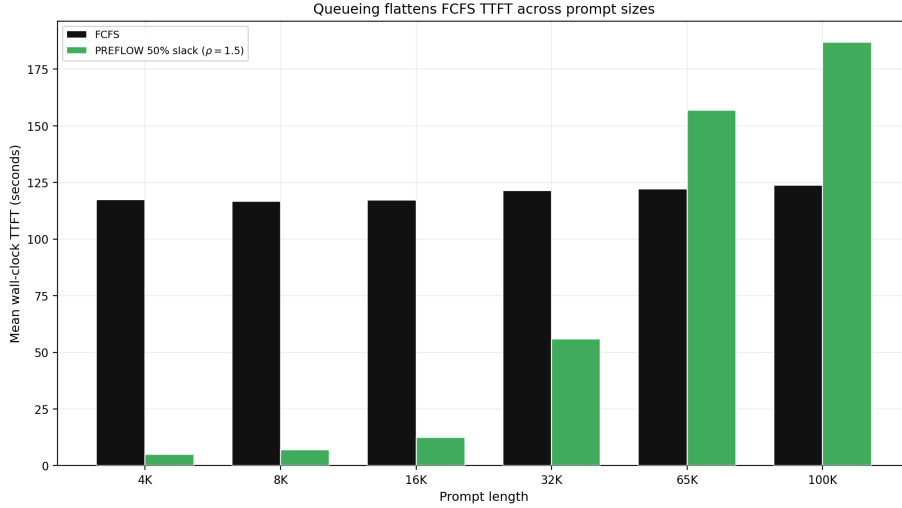


Figure 1: FCFS and PREFLOW-50 mean wall-clock TTFT by prompt length

All 11 policy configurations completed the same 1,000 requests with identical request IDs, prompt lengths, and scheduled arrival times. This is a controlled single-seed experiment, not a claim about every production workload. Its job is to expose the scheduling tradeoff cleanly.

Why aging is not a contract

Aging is a common way to soften starvation in work-aware scheduling. PrefillOnly is the representative aging baseline in this experiment: predicted work favors short requests, while a linear waiting-time term gradually raises the priority of requests left behind.

The coefficient λ controls how quickly waiting overcomes the work ranking. A small value stays close to shortest-work scheduling. A large value pushes the policy back toward FCFS. This creates an empirical efficiency/protection tradeoff, but λ does not specify a maximum per-request slowdown. The effective protection also changes with the arrival process, prompt distribution, chunk size, and hardware.

PREFLOW gives the operator a parameter with direct semantics instead. The parameter controls the maximum modeled inflation over the FCFS service a request could expect from the state it saw at arrival.

PREFLOW in three steps

PREFLOW makes one scheduling decision at each chunk boundary.

1. Freeze the request’s arrival-state FCFS baseline. For request i ,

B_i = predicted FCFS completion latency from the state at arrival.

This baseline includes unfinished work ahead of the request and counts any outstanding chunk exactly once. Later arrivals cannot enlarge it.

2. Convert that baseline into an immutable completion entitlement:

$$D_i = a_i + \rho B_i.$$

Here a_i is the logical arrival time. Setting $\rho = 1.3$ allows at most 30% modeled completion inflation over that request’s arrival-state FCFS baseline. The experiments use $\rho \in \{1.3, 1.5, 1.8, 2.2\}$.

3. Rank requests by the work-aware objective and execute the highest-ranked chunk that preserves every fixed deadline.

The implementation ranks safe requests by

$$\pi_i(t) = \frac{1}{p_i r_i(t)},$$

where p_i is the request’s isolated prefill work and $r_i(t)$ is its remaining work. This is the weighted-shortest-remaining-work index for total stretch. Stretch is useful when request sizes differ by orders of magnitude: an extra second is an $11\times$ completion time for a request that needs 0.1 seconds in isolation, but only $1.1\times$ for one that needs 10 seconds.

The score $1/(p_i r_i)$ is the weighted-shortest-remaining-work priority induced by the stretch objective. PREFLOW applies that priority only inside the feasible action set. If the highest-ranked chunk is unsafe, it tries the next candidate. An earliest-deadline action is the rescue choice when the queue has no slack for a preferred overtake.

Hard protection changes the benefit–harm tradeoff

The main comparison plots mean TTFT improvement against the largest observed matched-request slowdown. Farther right means lower mean TTFT; lower means less harm to the worst request. Lines connect measured parameter settings only. The plot contains no fitted or extrapolated policy curves.

The measured operating points are:

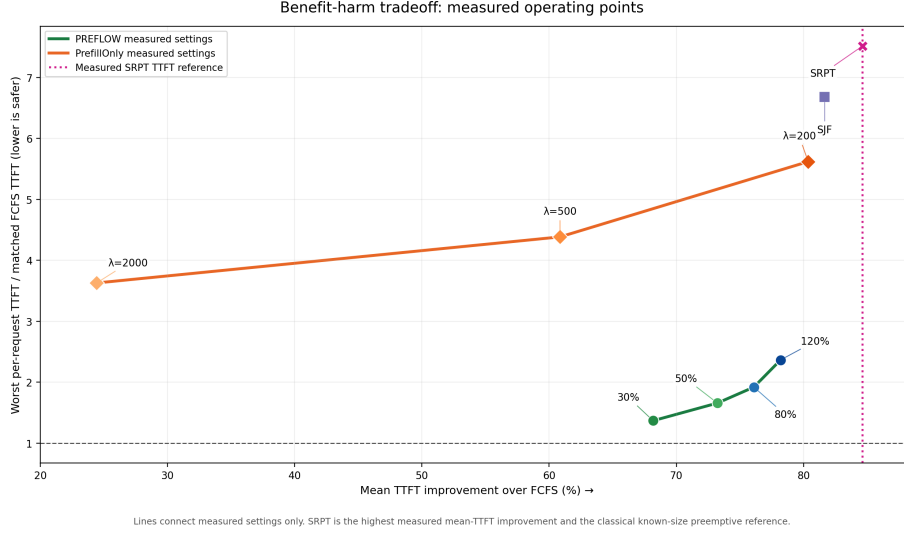


Figure 2: Measured TTFT benefit versus worst-request harm

Policy	Mean TTFT improvement over FCFS	Worst matched-request slowdown
SRPT	84.6%	7.52×
SJF	81.7%	6.68×
PrefillOnly $\lambda = 200$	80.4%	5.62×
PrefillOnly $\lambda = 500$	60.9%	4.38×
PrefillOnly $\lambda = 2000$	24.4%	3.63×
EDF 120%	5.8%	1.60×
PREFLOW 30%	68.2%	1.37×
PREFLOW 50%	73.2%	1.66×
PREFLOW 80%	76.1%	1.92×
PREFLOW 120%	78.2%	2.37×

The central comparison is PREFLOW against PrefillOnly. PREFLOW 120% gives up 2.2 percentage points of mean improvement relative to PrefillOnly $\lambda = 200$, while reducing the worst observed slowdown from $5.62\times$ to $2.37\times$. PREFLOW 30% is better on both measured axes than the most protective tested PrefillOnly setting: 68.2% versus 24.4% mean improvement and $1.37\times$ versus $3.63\times$ worst slowdown.

SRPT minimizes mean flow time in the classical known-size preemptive single-server model and gives the highest measured mean-TTFT improvement among the policies in this experiment. We therefore use it as the aggressive efficiency

reference, not as a formal bound for the real PD implementation. PREFLOW 120% retains about 92% of SRPT’s measured mean-TTFT improvement while cutting worst observed slowdown by more than threefold.

EDF is the ablation for the other half of the design. It uses the same frozen deadlines but always follows deadline order, removing PREFLOW’s greedy choice among safe actions. Its 5.8% mean improvement shows that deadlines provide protection, while the work-aware choice among safe actions produces most of the latency benefit.

These points describe the tradeoff measured on this workload and seed. They do not establish a complete Pareto frontier.

A hard modeled constraint produces a different tail

The slowdown tail answers the practical version of “why not tune aging?” A softer priority can make large slowdowns rarer. It cannot make the next unsafe overtake illegal.

There is one important distinction before reading the figure. PREFLOW enforces deadlines in charged, modeled work units inside a run. The plot divides observed wall-clock TTFT under PREFLOW by observed TTFT for the matched request in a separate FCFS execution. The numerator therefore includes cost-model error, asynchronous execution, admission effects, KV transfer, and run-to-run noise; the denominator also differs from the frozen internal baseline B_i .

For that reason, the measured endpoints are $1.369\times$, $1.658\times$, $1.920\times$, and $2.365\times$ for configured modeled bounds of $1.3\times$, $1.5\times$, $1.8\times$, and $2.2\times$. The figure evaluates how the modeled constraint translates to observed wall-clock latency. It is not a direct test of the theorem in its native units.

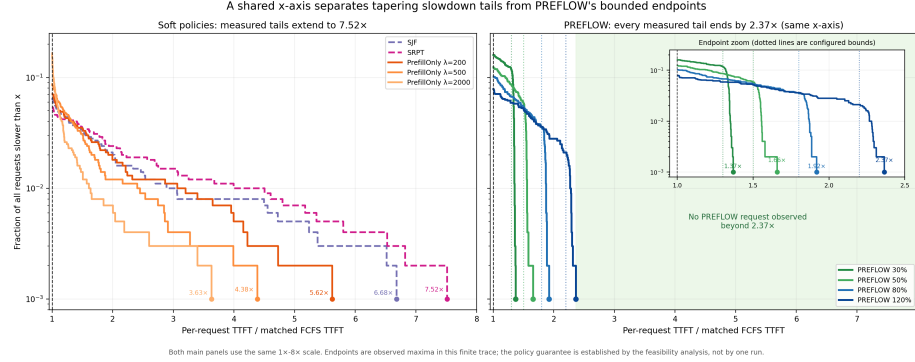


Figure 3: FCFS-relative slowdown tails

The horizontal axis is observed TTFT divided by matched FCFS TTFT. The vertical axis is the fraction of all requests slower than that ratio, on a logarithmic scale.

SJF, SRPT, and all three PrefillOnly settings produce tapering tails. Stronger aging shortens the observed tail, but every additional overtake remains a legal action. The endpoints from $3.63\times$ to $7.52\times$ are finite observations from this trace, not policy limits.

PREFLOW has a different shape. Each curve keeps a broad region of useful reordering and then ends near its configured FCFS-relative boundary. Once an earlier deadline has no prefix slack left, another bypass is illegal. Changing ρ moves that boundary; it does not change the rule that enforces it.

Queue behavior during bursts

Scheduling cannot create prefill compute. It can change which requests finish first, and that changes a request-count queue because short requests leave the system faster than long ones.

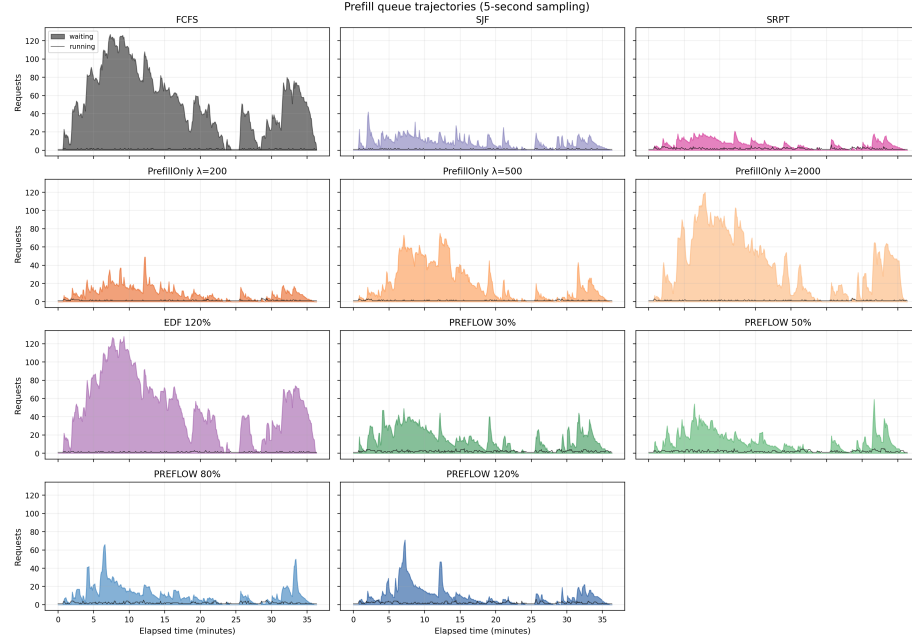


Figure 4: Prefill queue trajectories

FCFS carries 52.9 waiting requests on average and peaks at 128. EDF behaves similarly, averaging 49.7, because its cumulative FCFS-relative deadlines largely preserve arrival order. SRPT averages 6.5 waiting requests and peaks at 22 by draining short requests first, while PREFLOW 120% averages 9.7 with a $2.37\times$ rather than $7.52\times$ worst matched slowdown. Across the PREFLOW settings, mean waiting occupancy ranges from 15.0 at 30% slack to 9.7 at 120%.

PrefillOnly's queue is sensitive to λ : mean waiting occupancy grows from 9.2

requests at $\lambda = 200$ to 39.6 at $\lambda = 2000$. This is the same tuning tradeoff seen in latency.

Every panel returns to zero near the end of the run. The workload contains temporary bursts, but the system eventually processes all offered work. Lower queue depth here reflects completion order, not extra capacity or admission drops.

How the guarantee works

PREFLOW treats queue slack as an overtaking budget. At a chunk boundary, sort all unfinished released requests by fixed deadline. For the first k requests in that earliest-deadline-first (EDF) order, define

$$\sigma_k(t) = D_{(k)} - t - \sum_{j=1}^k r_{(j)}(t).$$

The deadline minus the current clock is the available service; the sum is the work the prefix still needs. A nonnegative σ_k means the prefix remains feasible.

For a candidate at EDF position m with next-chunk cost q , every earlier prefix loses q units of slack. Prefixes containing the candidate lose the same amount of time and remaining work, so their slack is unchanged. The chunk is safe exactly when

$$q \leq \min_{k < m} \sigma_k(t).$$

Prefix slack is the remaining overtaking budget. The earliest-deadline request has no earlier prefix, so it provides a rescue action whenever a preferred overtake is unsafe.

The arrival-state FCFS continuation supplies the initial feasible schedule. Accepted chunks preserve prefix feasibility, and later arrivals cannot change older deadlines. Under conservative cost and admission assumptions, request i therefore completes by $D_i = a_i + \rho B_i$. The full proofs are in the preprint source. The corresponding implementation remains the existing PREFLOWScheduler.

Implementation details and limits

Token-count and triangular attention proxies can be useful for ranking requests. They are insufficient for enforcing a meaningful completion-inflation contract: the feasibility test spends predicted action cost as time, so the charged units must track actual service time.

The reported PREFLOW results therefore use the startup-profiled device-time model. An empty prefill worker profiles the real model path and fits a monotone,

history-dependent predictor in milliseconds using the features $1, h, h^2$. Sparse measurements cover chunks smaller than the configured size. Summing these predictions gives isolated work, remaining work, and next-action cost.

Calibration requests and their KV state are removed before serving begins. The scheduling principle does not depend on this particular predictor; another additive device-time model could replace it without changing the deadline or prefix-slack rules.

KV admission adds another blocking point. PREFLOW reserves a conservative allowance for the resident set to drain; once a waiting request becomes admission-critical, younger work cannot enlarge its blocking set. The normal KV allocator and waiting/running lifecycle remain unchanged.

The theorem is exact in the scheduler’s charged model units. The current profiler fits median point estimates rather than conservative upper bounds, so it does not establish the same bound in wall-clock time. Charging an empirical upper quantile or measured error envelope would tighten that translation without redesigning the scheduler.

This evaluation also has a narrow scope: one synthetic trace and seed, dummy weights, one hardware topology, and no prefix caching. Real traces, prefix reuse, and other platforms remain to be evaluated. The guarantee applies to prefill service under the stated model and admission assumptions.

The prefill advantage remains visible end to end

Decode adds another stage after the scheduler-controlled prefill. The same benefit/harm separation remains visible when we measure full end-to-end latency for the 128-token outputs.

PREFLOW spans 66.6% to 76.7% mean E2E improvement, with worst observed E2E slowdown from $1.43\times$ to $2.30\times$. PrefillOnly $\lambda = 200$ reaches 78.8% mean improvement with $5.38\times$ worst slowdown. SRPT reaches 83.4% and $6.95\times$.

These are measured two-stage results. PREFLOW provides a prefill completion guarantee; it does not provide a full E2E guarantee for downstream decode.

Conclusion

Preemption fixes head-of-line blocking by moving some requests ahead of others. The unresolved question is how much delay those overtakes may impose on the requests left behind.

Aging modifies the priority and relies on tuning to shape the resulting tail. PREFLOW constrains the legal schedule. The work-aware objective still chooses the highest-priority next action, but only inside a safety envelope that preserves every fixed FCFS-relative entitlement already in the system.

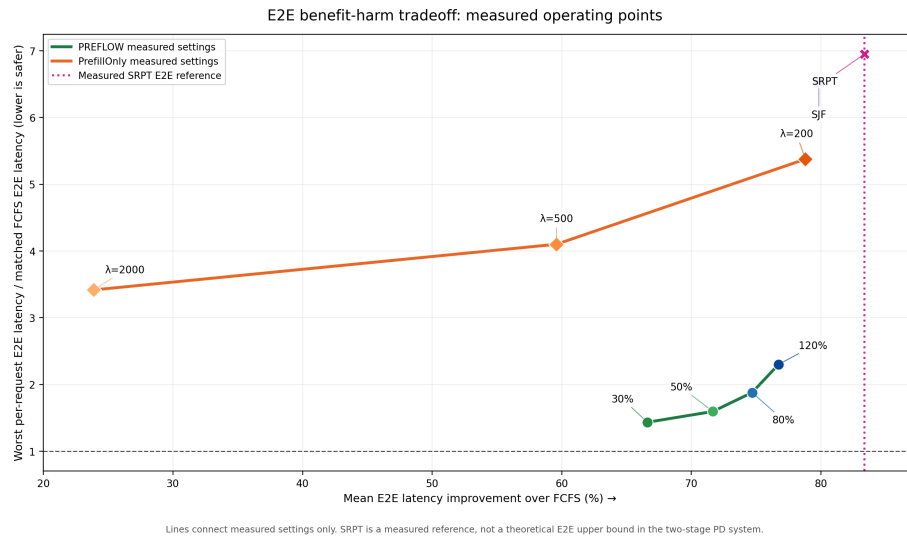


Figure 5: Measured E2E benefit versus worst-request harm